ELF-DOS Developer's Guide

This guide explains how ELF-DOS programs are built, how the kernel API works, and how to write new programs and commands. For day-to-day use of the finished system, see the *ELF-DOS User's Guide*.

Programs are written in CDP1802 assembly language. This guide assumes you already know that language; it documents ELF-DOS's own conventions on top of it - how a program is loaded and started, what each kernel call does, and the register usage each one expects. Register names `R0` through `RF` refer to the CDP1802's sixteen 16-bit registers (`RA` through `RF` stand for `R10` through `R15`); `D` is the 8-bit accumulator, and `DF` is the 1-bit carry/borrow flag most calls use to report success or failure.

The Toolchain

Programs are built with two tools: `asm02`, the assembler, and `link02`, the linker. Both understand a small set of extensions beyond plain 1802 assembly - a synthetic-opcode mechanism (`.op`, used to define multi-instruction pseudo-ops like `MOV` and `CALL`) and a short-branch relaxation pass (`-r`, which shrinks a long branch to a short one wherever the target is close enough) - that are specific to this toolchain, not standard across every 1802 assembler.

Program Binary Format and Calling Convention

Every program starts with the same six-byte header the kernel itself uses:

```
Offset   Contents
0-2      The letters 'E', 'D', 'F'
3        Program version number
4-5      Reserved
6+       Program code -- this is where execution starts
```

Programs are loaded at a fixed address, `PROG_BASE`, defined in `kernel_api.inc`.

At entry:

- `RA` - a pointer to the argument list (`argv`): an array of 16-bit, big-endian pointers, one per argument, each pointing to a plain, null-terminated piece of text. The first argument, `argv[0]`, is the program's own invocation name, matching the C convention `main(argc, argv)`.
- `RC` - the number of arguments (`argc`). Always at least 1.
- `R2` - the stack pointer, already set up and usable.

Every other register, and both `D` and `DF`, start out with an unknown value.

`RA` and `RC` are only good until the program's first kernel or BIOS call. After that, either one may have changed, since nothing guarantees a register survives a call unless that call's own documentation says so. Copy whatever you need out of them right away. A minimal skeleton:

```
#include    include/kernel_api.inc

            org     PROG_BASE
            db      'E','D','F'         ; executable signature
            db      1,0,0               ; program version number

start:
            ghi     ra                  ; argv is a table of pointers;
            phi     rf                  ; argv[1] is the first real
            glo     ra                  ; argument (argv[0] is this
            plo     rf                  ; program's own name)
            inc     rf
            inc     rf
            lda     rf
            phi     rd
            ldn     rf
            plo     rd                  ; RD = argv[1]

            ; ... use RD as a pointer to the first argument's text ...

            ldi     0                   ; exit code 0 = success
            rtn
```

To end the program, return with `D` set to an exit code: 0 for success, anything else is program-defined. A batch script or another program can read this value back through `K_GET_ERRORLEVEL` (see below).

A program's usable memory - where it can safely put a heap, for example

- is given at a fixed location, `LOADER_ARGS`, not in a register: word 0 is `mem_base`, word 1 is `mem_top`. Read this directly if the program needs to know its own memory bounds, for example to size a heap.

Writing a New Program

A few patterns come up often enough to be worth using rather than reinventing:

- **One required argument**, such as a file name. Check that `argc` is at least 2, then read `argv[1]` (shown above).
- **Several independent arguments**, such as deleting more than one file in a single command. Loop over the argument list, handle each one on its own, and if one fails, print an error for it and keep going rather than stopping the whole command — several of the built-in commands (deleting files, copying files, changing attributes) all work this way, and stay quiet when everything succeeds.
- **Wildcards.** A library module provides pattern matching that can be paused and resumed one match at a time (see "Library Modules" below). If a wildcard matches nothing, fall back to using the text exactly as typed, rather than reporting an error.
- **Paths.** Most file and directory calls already understand drive letters and relative paths on their own; hand them the path text directly rather than resolving it yourself first.
- **A heap.** One provided library module is a simple, fast allocator with no way to free a single item, good for "collect everything, use it, then exit" programs. Another is a general-purpose allocator with

a real `free`. Both need to be told the program's memory bounds once, by calling their own `init` with the values read from `LOADER_ARGS`.

Kernel API Reference

Every kernel call is reached through a fixed jump table, so a program never depends on the kernel's own internal layout. A call is made the same way as any subroutine call, with arguments in the registers listed below:

```
            ldi     0                   ; mode 0 = read
            call    K_FILE_OPEN
```

New calls are only ever added to the end of the table, never reordered or removed, so a program built against an older kernel keeps working after the kernel is rebuilt.

Unless stated otherwise, a call's `DF` result follows the usual convention: `DF = 0` means success, `DF = 1` means failure. A register not listed under "Returns" should be assumed changed by the call and not relied on afterward.

Working with files

Every open file is represented by a File Control Block (FCB) that the *caller* owns and allocates; there is no kernel-imposed limit on how many files a program can have open at once, only how much memory it is willing to spend. To open a file, a program reserves two blocks of its own memory:

```
my_fcb:     ds      FCB_LEN         ; 32 bytes -- need not be pre-zeroed
my_iobuf:   ds      FCB_IOBUF_LEN   ; 512 bytes, this FCB's own sector
buffer
```

and passes pointers to both to `K_FILE_OPEN`. Every later call that touches this file — `K_FILE_CLOSE`, `K_FILE_READ`, `K_FILE_WRITE`, `K_FILE_SEEK` — takes the FCB pointer directly; there is no separate small-integer handle to keep track of. The internal layout of an FCB is not documented here and should not be relied on. Treat it as an opaque block the kernel manages on the caller's behalf.

`K_FILE_OPEN` Opens a file for reading, or for reading and writing.

- **Args:** `RF` = path, `D` = mode (0 = read, 1 = read/write, creating the file if it doesn't already exist), `RD` = pointer to the caller's `FCB_LEN`-byte FCB, `RA` = pointer to the caller's `FCB_IOBUF_LEN`-byte I/O buffer.
- **Returns:** `DF` = 0/1. `D` is not meaningful on return. The caller already has the FCB pointer it passed in.

`K_FILE_CLOSE` Closes a file previously opened with `K_FILE_OPEN`.

- **Args:** `RD` = the same FCB pointer passed to `K_FILE_OPEN`.
- **Returns:** `DF` = 0/1.

`K_FILE_READ` / `K_FILE_WRITE` Reads or writes bytes at the file's current position, advancing it by the number of bytes actually transferred.

- **Args:** `RD` = FCB pointer, `RF` = data buffer, `RC` = byte count.
- **Returns:** `RC` = bytes actually transferred (read only - a short count usually means end of file), `DF` = 0/1.

`K_FILE_SEEK` Moves a file's current position without reading or writing any data. File positions, offsets, and sizes are tracked as full 32-bit values.

- **Args:** `RD` = FCB pointer, `RC` (low byte) = whence (0 = from the start of the file, 1 = relative to the current position, 2 = relative to the end of the file), `RA:R9` = signed 32-bit offset (`RA` = high word, `R9` = low word).
- **Returns:** `DF` = 0 on success, `RD` = the resulting position (low 16 bits only - there is no way to retrieve the high 16 bits from this call). `DF` = 1 if the whence value is invalid, the resulting position would fall outside the file, or an I/O error occurred; the file's position is left unchanged in that case.

`K_FILE_DELETE` Deletes a file. Refuses to delete a directory.

- **Args:** `RF` = path.
- **Returns:** `DF` = 0/1 (not found, is a directory, or an invalid path component are all errors).

`K_FILE_RENAME` Renames a file or directory. The new name must stay within the same parent directory - this call cannot move something to a different directory.

- **Args:** `RF` = old path, `RD` = new name (a bare name, no path separators).
- **Returns:** `DF` = 0/1 (the old path not existing, the new name already existing, or either name being `.`/`..` are all errors).

`K_FILE_TOUCH` Updates a file or directory's last-write date and time to right now. Nothing else about it changes.

- **Args:** `RF` = path.
- **Returns:** `DF` = 0/1.

`K_FILE_SETATTR` Changes a file or directory's attribute byte, by masking it: bits in the "set" mask are turned on, bits in the "clear" mask are turned off, and this is a general-purpose call, not limited to one particular attribute.

- **Args:** `RF` = path, `RC` (low byte) = bits to set, `RC` (high byte) = bits to clear. The new attribute byte is `(old & ~clear) | set`.
- **Returns:** `DF` = 0/1.

`K_STAT` Looks up a file or directory without opening it, filling in the same result format `K_DIR_READ` uses (see "Directories" below). Works on either a file or a directory.

- **Args:** `RF` = path, `RD` = pointer to a caller-provided `DIRENT_LEN`-byte buffer.
- **Returns:** `DF` = 0 on success (buffer filled), `DF` = 1 if not found or the path is invalid.

Directories

`K_DIR_OPEN` Begins a directory listing.

- **Args:** RD = the directory's own starting cluster (0 means the root directory).
- **Returns:** nothing meaningful — a following K_DIR_READ starts from the beginning.

K_DIR_READ Returns the next entry in a directory listing started by K_DIR_OPEN.

- **Args:** RD = pointer to a caller-provided DIRENT_LEN-byte result buffer.
- **Returns:** DF = 0 with the buffer filled in if an entry was available, DF = 1 at the end of the directory. The result buffer has this layout:

Field	Offset	Size	Contents
DIRENT_NAME	0	up to 127 chars	Null-terminated file name.
DIRENT_ATTR	128	1	Attribute byte — see ATTR_DIR/ATTR_HIDDEN below.
DIRENT_CLUST	129	2	First cluster, big-endian.
DIRENT_SIZE	131	4	File size in bytes, big-endian.
DIRENT_WRTTIME	135	2	Last-write time, big-endian, packed: bits 15-11 = hour, 10-5 = minute, 4-0 = seconds/2.
DIRENT_WRTDATE	137	2	Last-write date, big-endian, packed: bits 15-9 = year - 1980, 8-5 = month, 4-0 = day.

DIRENT_LEN (139) is the total buffer size to declare. ATTR_DIR ($10) is set in DIRENT_ATTR for a subdirectory; ATTR_HIDDEN ($02) is set for a hidden entry.

K_DIR_SAVE_STATE / **K_DIR_RESTORE_STATE** K_DIR_OPEN/K_DIR_READ share one scan position, so only one directory listing can be "in progress" at a time. These two calls let a program pause a listing, do something else (open a file, look something else up), and resume the listing exactly where it left off.

- **K_DIR_SAVE_STATE** — **Args:** RF = pointer to a caller-owned DIR_STATE_LEN-byte buffer. **Returns:** nothing; the buffer is filled in.
- **K_DIR_RESTORE_STATE** — **Args:** RF = pointer to a buffer previously filled by K_DIR_SAVE_STATE. **Returns:** DF = 0 on success (the next K_DIR_READ resumes from the snapshot); DF = 1 if the disk could not be re-read to restore the scan position - treat this the same as any other I/O error.

K_DIR_CREATE Creates a new, empty subdirectory. Single-level only; the parent directory must already exist.

- **Args:** RF = path.
- **Returns:** DF = 0/1 (already exists, an invalid path component, or a full disk are all errors).

K_DIR_REMOVE Removes an empty subdirectory. Refuses a directory that still has entries in it, `.`, `..`, or the root directory itself.

- **Args:** RF = path.
- **Returns:** DF = 0/1.

Paths and drives

`K_PATH_RESOLVE` Resolves a path — optionally prefixed with a drive letter (`C:` through `F:`) — into a parent directory and a final component, without looking up the final component itself. Every component before the last one must already be a real directory (`.` and `..` work automatically, since they are ordinary directory entries). This is the same resolution logic every path-based file and directory call already uses internally. Call it directly only when you need the pieces separately, for example to decide whether the final component should name a file or a directory.

- **Args:** `RF` = path (not modified).
- **Returns:** `RD` = the resolved parent directory's cluster. `RF` = pointer to the final path component, null-terminated. This points into kernel-owned scratch memory and stays valid only until the next `K_PATH_RESOLVE` call. Empty if the path ended in a separator or was just `/` (in that case, the cluster in `RD` *is* the target). `RC` (low byte) = the resolved drive index (0-3, 0 = `C:`). `DF` = 0 on success, `DF` = 1 if an intermediate component wasn't found or wasn't a directory, or an explicit drive prefix named a drive with nothing mounted on it.

`K_GETCURDIR` / `K_SETCURDIR` Each drive remembers its own current directory independently, separate from which drive is currently active — the same convention classic MS-DOS uses. Changing a drive's remembered directory does not switch to that drive.

- `K_GETCURDIR` — **Args:** none. **Returns:** `RD` = the active drive's current directory cluster (0 = root), `D` = the active drive index (0-3).
- `K_SETCURDIR` — **Args:** `D` = drive index (0-3), `RD` = new current-directory cluster for that drive. **Returns:** nothing.

`K_SETDRIVE` Switches which drive is active.

- **Args:** `D` = drive index (0-3) to make active.
- **Returns:** `DF` = 0 on success, `DF` = 1 if that drive has nothing mounted (nothing changes in that case).

`K_GETSHELLDRIVE` Reports which drive the command shell itself was found on at boot — almost always drive 0 (`C:`) — for use as a fallback location when looking for a command.

- **Args:** none.
- **Returns:** `D` = that drive's index (0-3).

Console input and output

`K_TYPE` Writes one character to the console.

- **Args:** `D` = the character.
- **Returns:** nothing meaningful.

`K_MSG` Writes a null-terminated string, pointed to by a register, to the console.

- **Args:** `RF` = pointer to the string.
- **Returns:** nothing meaningful.

`K_INMSG` Writes a null-terminated string that immediately follows the call instruction itself, rather than being pointed to by a register - convenient for a short literal message:

```
        call    K_INMSG
        db      "Hello.",13,10,0
```

- **Args:** none (the text follows the call in the code itself).
- **Returns:** nothing meaningful; execution continues right after the null byte.

`K_INPUTL` Reads one line of input.

- **Args:** `RF` = destination buffer, `RC` = its maximum length.
- **Returns:** `DF` = 0 with a real line in the buffer. `DF` = 1 only if input has been redirected from a file and that file has run out. Live keyboard input never returns `DF` = 1, since there is no "end of file" from a keyboard. Check `DF` if you loop on this call, so a program reading from an exhausted redirected input doesn't spin forever.

`K_READ` Reads a single character, blocking until one is available. Aware of input redirection the same way `K_INPUTL` is.

- **Args:** none.
- **Returns:** `D` = the character read.

`K_TTY` A direct passthrough to the console's own single-character output routine, bypassing any output redirection. Rarely needed; `K_TYPE` is the ordinary choice.

- **Args:** `D` = the character.
- **Returns:** nothing meaningful.

`K_GETDEV` Reports which peripheral devices the BIOS detected at boot (for example, whether a real-time clock is present).

- **Args:** none.
- **Returns:** device flags in `D` - see the BIOS's own documentation for the bit layout, which this call passes through unchanged.

Clock

`K_GETTOD` / `K_SETTOD` Read or set the time-of-day clock, if the hardware has one.

- **Args (`K_SETTOD`):** see the BIOS's own time-of-day format.
- **Returns:** the current time-of-day value, in the same format.

Raw disk access

These two calls bypass the file system entirely and address the disk directly by sector number. **Use them with real care** - a mistake here can corrupt the file system or the disk's own boot sectors. They exist for the rare program (a disk-label editor, an installer) that needs to touch a specific on-disk byte with no ordinary file-system call to reach it.

`K_SECREAD` / `K_SECWRITE` Reads or writes one raw 512-byte sector by its logical block address.

- **Args:** R7/R8 = the 24-bit sector address (R8 low byte = bits 23-16, R7 high byte = bits 15-8, R7 low byte = bits 7-0, R8 high byte = 0), RF = pointer to a 512-byte buffer (the data to write, or where to put what's read).
- **Returns:** DF = 0/1. R7/R8 are not preserved across the call.

Memory

K_HIMEM_RESERVE / **K_HIMEM_RELEASE** General-purpose reservation of a block of high memory, for anything that needs temporary space beyond its own normal allocation, such as loading a relocatable module, for example. Pure mechanism: no flags, no automatic tracking: the caller is responsible for releasing exactly what it reserved.

K_HIMEM_RESERVE

- **Args:** RC = bytes to reserve.
- **Returns:** DF = 0 with RD = the reservation's base address. DF = 1 if there isn't enough headroom (nothing changes in that case).

K_HIMEM_RELEASE

- **Args:** RC = bytes to release — must match a prior successful K_HIMEM_RESERVE call exactly.
- **Returns:** nothing.

K_GLOB_RESERVE Reserves a fixed-size block of high memory for wildcard-expansion use. Safe to call more than once — a later call while already reserved just returns the same address again, with no additional effect.

- **Args:** none.
- **Returns:** DF = 0 with RD = the reservation's base address (GLOB_BUF_LEN bytes usable from there). DF = 1 if there isn't enough headroom.

Miscellaneous

K_GET_ERRORLEVEL Reads back the exit code of the last command that ran.

- **Args:** none.
- **Returns:** D = the last exit code (0-255), DF = 0 always.

Constants Worth Knowing

Name	Value	Meaning
PROG_BASE	(see kernel_api.inc)	The fixed address every program loads to.
LOADER_ARGS	PROG_BASE - 4	Word 0 = mem_base, word 1 = mem_top — the program's usable memory range.
FCB_LEN	32	Size of a File Control Block a program must allocate for each open file.
FCB_IOBUF_LEN	512	Size of the I/O buffer that goes with each FCB.

Name	Value	Meaning
`DIRENT_LEN`	139	Size of the result buffer `K_DIR_READ` and `K_STAT` fill in.
`DIR_STATE_LEN`	9	Size of the snapshot buffer `K_DIR_SAVE_STATE`/`K_DIR_RESTORE_STATE` use.
`ATTR_DIR`	`$10`	`DIRENT_ATTR` bit for a subdirectory.
`ATTR_HIDDEN`	`$02`	`DIRENT_ATTR` bit for a hidden entry.
`GLOB_BUF_LEN`	768	Size of `K_GLOB_RESERVE`'s own reservation.

Library Modules

A handful of library modules are provided alongside the kernel API, ready to link into a program that needs them. None of them are stand-alone programs; they have no header of their own, and are assembled separately and linked into whichever program wants to use them.

Module	What it provides
`env.asm`	Reading, setting, and removing environment variables.
`file_glob.asm`	Wildcard (`*`/`?`) matching that can be paused and resumed one match at a time.
`fmt32.asm`	Formatting a large (32-bit) number with comma grouping.
`heap_bump.asm`	A simple, fast memory allocator with no per-item `free`.
`heap_malloc.asm`	A general-purpose allocator, with `free` and coalescing of freed blocks.
`icall.asm`	Safely calling through an address that is only known while the program is running.
`lineedit.asm`	Cursor movement and editing on a typed line — arrow keys, Home/End, and so on.
`modload.asm`	Loading a relocatable module at whatever address is currently free.
`move.asm`	Renaming a file where possible, falling back to copy-then-delete otherwise.
`pathstr.asm`	Turning a directory's starting cluster back into a full path string.
`vollabel.asm`	Reading and writing a drive's volume label.
`ymodem.asm`	The YMODEM file-transfer protocol.